

Lab 4.1: Switches, LEDs, 7-Seg Mux

Microprocessors (010153521) | Semester 1/2026

Duration: 3 hours (in-lab) | **Topic:** Breadboard I/O shield for the Arduino Uno R3 (push buttons, LEDs, and a multiplexed 7-segment display), tested with AVR assembly

Objectives

- Design a general I/O circuit that fits inside the Arduino Uno's pin budget by **time-division multiplexing** a 2-digit 7-segment display, instead of wiring one display driver pin per segment per digit.
- Wire push-button inputs with the ATmega328P's **internal pull-up** resistors (Lecture 4, §4.2-4.3) and individually addressable LED outputs with **SBI/CBI** (Lecture 4, §4.4-4.5) on a breadboard, driven by the same Arduino Uno R3 toolchain as Lab 3.
- Write and test AVR assembly that scans buttons with **SBIC/SBIS**, converts a digit to a 7-segment pattern, and refreshes a multiplexed display fast enough to avoid visible flicker – applying Lecture 3's cycle-counting delay methods to a real refresh-rate budget.
- Produce a verified, working breadboard circuit and firmware – this is the exact circuit you will capture into a KiCad schematic and PCB in **Lab 4-2**.

Quick Reference: Pin Assignment

Every signal fits inside the 18 pins Lab 3 already established as usable on the Arduino Uno R3 (ATmega328P) – PB0–PB5, PC0–PC5, and PD2–PD7 (PD0/PD1 stay off-limits, reserved for USB-serial).

Port bits	Uno pins	Direction	Function
PB0–PB3	D8–D11	Input, pull-up	SW1–SW4 push buttons (active low : pressed = 0)
PB4, PB5	D12, D13	Output	DIG1, DIG2 digit-select transistor bases (D13 doubles as the Uno's on-board LED – see Try It 1)
PC0–PC5	A0–A5	Output	Segments a,b,c,d,e,f (shared by both digits)
PD2	D2	Output	Segment g (shared by both digits)
PD3–PD7	D3–D7	Output	LED1–LED5, individually addressable

Segments **a–f** land on PORTC bits 0–5 in the same bit order used by the standard segment code table below, so one OUT PORTC,Rd sets all six at once – only segment **g** (on PORTD) needs a separate SBI/CBI.

Quick Reference: 7-Segment Codes (Common Cathode)

Both digits are common-**cathode**: a segment lights when its drive pin is **HIGH** (current flows pin → resistor → segment LED → common cathode). Bit 0 = segment **a** ... bit 5 = segment **f**, bit 6 = segment **g** (bit 7 unused) – exactly the classic **gfedcba** encoding.

Digit	0	1	2	3	4	5	6	7	8
Hex	0x3F	0x06	0x5B	0x4F	0x66	0x6D	0x7D	0x07	0x7F

Digit 9 = 0x6F. Bits 0–5 of each byte go straight to PORTC; bit 6 (segment **g**) must be tested separately (e.g. SBRC/SBRS) and mirrored onto PD2 with SBI/CBI.

Quick Reference: Multiplexing and Refresh Rate

Only *one* digit's common cathode is ever grounded at a time – the other digit's segments have no return path, so only the grounded digit actually lights, even though both digits' segment pins are wired in parallel. Show each digit for a short **dwel time** t_{dwell} , then switch to the other – fast enough that the human eye's flicker-fusion threshold ($\approx 50\text{--}60\text{ Hz}$) blends the two into one steady image:

$$f_{\text{refresh}} = \frac{1}{N_{\text{digits}} \times t_{\text{dwell}}}, \quad N_{\text{digits}} = 2 \Rightarrow t_{\text{dwell}} \leq \frac{1}{2 \times 60\text{Hz}} \approx 8.3\text{ ms}$$

Stay comfortably under this bound – a t_{dwell} near the limit will show visible flicker on real hardware even though it looks fine calculated on paper.

Quick Reference: Your Individual Target

As in Lab 3, derive your personal targets from your student ID before touching the keyboard:

D = sum of every digit in your student ID

Exercise	Your target
3 (multiplexed display)	$t_{\text{dwell}} = 2\text{ ms} + (D \bmod 8) \times 0.5\text{ ms}$
4 (button counter)	$N_0 = D \bmod 100$ (power-on displayed value)

Worked example (a made-up ID, do not reuse): ID 1029384756123 $\rightarrow D = 51$. Then $t_{\text{dwell}} = 2 + (51 \bmod 8) \times 0.5 = 2 + 3 \times 0.5 = 3.5\text{ ms}$ (refresh rate $\approx 143\text{ Hz}$, well clear of the 60 Hz bound), and $N_0 = 51 \bmod 100 = 51$.

Bill of Materials

Qty	Part	Notes
1	Arduino Uno R3 (from Lab 3)	Supplies power, ground, and all 18 signal pins
4	Tactile push button (6 mm THT)	No external resistor – internal pull-ups only
5	LED, 3 mm or 5 mm	Any color; one per LED1–LED5
5	Resistor, 330 Ω	Current limiting for LED1–LED5
2	7-segment display, common cathode	DIGIT1 (tens), DIGIT2 (units)
7	Resistor, 330 Ω	Shared segment resistors a–g (one per segment, not per digit)
2	NPN transistor (e.g. 2N3904)	Digit-select low-side switches
2	Resistor, 1 k Ω	Transistor base resistors
1	Breadboard + jumper wires	

Building the Circuit

- Buttons.** Wire one leg of each SW1–SW4 to PB0–PB3 (Uno D8–D11), the other leg to **GND**. Do *not* add an external pull-up or pull-down – firmware will enable the AVR's internal pull-up, so a pressed button reads 0 and a released one reads 1.
- LEDs.** Wire each of LED1–LED5 anode $\rightarrow$ 330 Ω resistor $\rightarrow$ PD3–PD7 (Uno D3–D7); cathode $\rightarrow$ **GND**.
- Segment bus.** For each of the 7 segments (a–g), tie *both* digits' same-lettered segment pin together, then run that shared node through one 330 Ω resistor to the matching MCU pin (a–f $\rightarrow$ PC0–PC5; g $\rightarrow$ PD2). This is the heart of the multiplexing trick: 7 resistors and 7 MCU pins drive *both* digits, not 14.

4. **Digit-select transistors.** DIGIT1's common-cathode pin goes to Q1's **collector**; Q1's **emitter** goes to GND; Q1's **base** goes through a 1 k Ω resistor to PB4 (D12). Wire DIGIT2 to Q2 the same way, based from PB5 (D13). Driving PB4/PB5 HIGH saturates the transistor, sinking that digit's entire segment current to ground – the AVR pin only supplies a few mA of *base* current, not the segments' combined current.

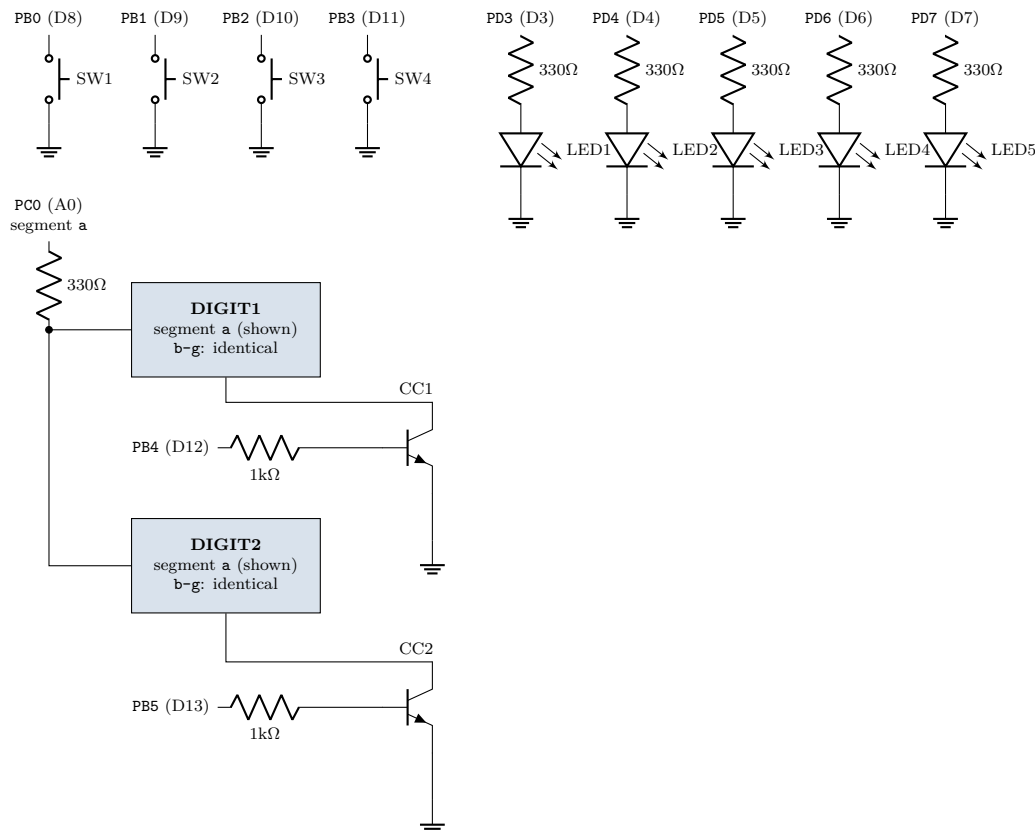


Figure 1: Full schematic for buttons (SW1-SW4) and LEDs (LED1-LED5), plus the multiplexed-display driver with segment **a** drawn in full – segments **b-g** repeat the identical pattern (own 330 Ω resistor from PC1-PC5/PD2, tied to both digits' matching pin) and are omitted here only to keep the figure readable. CC1/CC2 are each digit's common cathode; only one of Q1/Q2 conducts at a time, so only one digit's segments have a path to ground at any instant.

Quick Experiments (Try It)

Try It 1: The on-board LED is watching PB5

Before wiring anything else, load Lab 3's blink skeleton (or any program that toggles PB5) and confirm the Uno's on-board D13 LED blinks. Now wire DIG2 (Exercise 3 below) to PB5 as instructed above and re-run your multiplexed display firmware.

1. Does the on-board LED flicker at the same rate as DIGIT2's refresh?
2. Explain why sharing a pin between the on-board LED and DIG2's transistor base does *not* stop the display from working – what does the on-board LED's own series resistor mean for how much current it steals from the transistor's base?

Try It 2: One digit, no multiplexing

Wire *only* DIGIT1 (leave DIGIT2 unpowered, PB5 low). Write a short loop that walks digit values 0 through 9, holding each for about half a second, using the segment code table above and a single `OUT PORTC + SBI/CBI PORTD, 2` per digit. Confirm every digit 0-9 renders exactly as expected before moving on to multiplexing two digits at once – debugging one static digit is far easier than debugging two flickering ones.

In-Lab Exercises (target: 3 hours)**In-Lab Exercise 1: Buttons and LEDs (35 min)**

Wire SW1–SW4 and LED1–LED5 per the instructions above. Write and run a program that mirrors each button directly to one LED (e.g. SW1 → LED1), using the SBIC/SBIS polling pattern from Lecture 4’s “Reading a Single Bit: Switch to LED” example.

- Confirm DDRB bits 0-3 are **0** (input) and PORTB bits 0-3 are **1** (pull-up enabled) – verify with IN/inspect in the simulator or by reasoning through your `.asm`, not just by observing the LED behave correctly (a swapped polarity can look right by accident on some button wiring).
- Record, in your notebook, what LED1 does the instant power is applied, *before* you press anything – and explain why, in terms of the pull-up’s default state.

In-Lab Exercise 2: Static digit and the segment table (30 min)

Repeat Try It 2 as a graded checkpoint: DIGIT1 alone, cycling 0-9 with a visible hold time per digit. Write a `DIGIT_TO_SEG` subroutine (input: a digit 0-9 in a register; output: the segment byte in another register) using a CPI/BRNE comparison chain – *not* a lookup table – so each comparison is explicit and traceable in your `.lst` file.

In-Lab Exercise 3: Multiplexed 2-digit counter (55 min)

Using your t_{dwell} from the “Your Individual Target” box, build the full multiplexed **REFRESH** loop: show the tens digit for t_{dwell} with DIG1 selected, then the units digit for t_{dwell} with DIG2 selected, repeating forever. Drive a free-running two-digit count (00-99, wrapping) that increments roughly once per second, independent of the display refresh loop.

1. Show your cycle accounting for t_{dwell} (the delay subroutine tuned to your target, same method as Lab 3) and your calculated f_{refresh} .
2. Build it. Does the display look like one steady image, or can you see it flicker or “strobe” when you wave a hand or pencil quickly in front of it? If you see flicker, is your measured behavior consistent with your calculated f_{refresh} against the 60 Hz bound?
3. **Trace/predict first, then verify:** before running it, predict what the display shows if you accidentally swap the DIG1/DIG2 select lines but leave the tens/units segment data unswapped. Then swap them on purpose and confirm.

In-Lab Exercise 4: Button-controlled counter (40 min)

Using your N_0 from the “Your Individual Target” box, initialize the displayed value to N_0 at power-on. Wire SW1 to increment and SW2 to decrement the displayed 2-digit value (wrapping 99→00 and 00→99), each on a single button press – not continuously while held.

- A mechanical button “bounces” – its contact opens and closes several times over a few milliseconds before settling. Add a simple software debounce (a short delay after detecting

a press, then re-checking the pin is still low) and describe in your notebook what you observed *without* it (e.g. the count jumping by more than 1 per press).

- Confirm the power-on value matches your N_0 exactly before testing the buttons.

AI Collaboration

Try these prompts with an AI tool:

- “I’m multiplexing a 2-digit common-cathode 7-segment display from an AVR. What dwell time per digit keeps me safely under the flicker-fusion threshold, and how does that change with more digits?”
- “How do I use SBIC/SBIS to poll a push button wired with an internal pull-up, and what’s a simple software debounce technique in AVR assembly?”
- “What’s the difference between common-cathode and common-anode 7-segment segment codes, and how do I convert one table into the other?”

Critical check – verify, don’t just accept: If an AI tool gives you a segment code table, check every byte against the Quick Reference table above bit by bit – segment-code tables online are inconsistent about bit order (*gfedcba* vs. *dp g f e d c b a*) and about common-cathode vs. common-anode polarity, and a wrong assumption here will show a plausible-looking but wrong digit on real hardware.

Know the limit: AI tools are prone to computing your t_{dwell} delay-loop cycle count without accounting for the REFRESH loop’s own overhead – the DIGIT_TO_SEG comparison chain and the SBI/CBI for segment *g* all eat into your dwell-time budget, just like RET overhead did in Lab 3. A plausible-looking calculation can still miss the real f_{refresh} by enough to cause visible flicker – verify on the real hardware, not just on paper.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Refresh rate at a different dwell time

A different student’s design uses a *4-digit* multiplexed display (not this lab’s 2-digit design) with $t_{\text{dwell}} = 3$ ms per digit. Calculate f_{refresh} and state whether it is above or below the 60 Hz flicker-fusion bound used in this lab – would you expect visible flicker?

Find the Bug 1: Segment bus tied to the wrong port

A student wired segments *a–f* to PC0–PC5 correctly, but wired segment *g* to PD3 instead of PD2 (an easy off-by-one on a breadboard) – and then wrote firmware that (correctly, per the lab’s instructions) sets/clears PD2 for segment *g*. DDRD and PORTD were otherwise configured exactly as this lab specifies for LED1–LED5. Predict exactly what appears on the display (both digits, all ten digit values) and separately what happens to LED1, and explain both in terms of what pin the firmware is actually toggling versus what pin segment *g* is actually wired to.

Write Code on Paper 1: Segment code for a common-anode display

On paper, without a computer, simulator, or AI tool, rederive the segment byte for digits 0, 5, and 8 if DIGIT1/DIGIT2 were common-**anode** displays instead of common-cathode

(segment lights when its drive pin is **LOW**, and the “common” pin is tied to **+5V** through the digit-select transistor instead of to GND). State whether the digit-select transistor type (NPN vs. PNP) would also need to change, and why.

Draw on Paper 1: Predict, then verify, the multiplexed waveform

Before building Exercise 3, sketch on paper the voltage you expect at PB4 (DIG1 select) and PB5 (DIG2 select) on the same time axis: axes labeled, both logic levels marked, and t_{dwell} and the overall refresh period $2 \times t_{\text{dwell}}$ labeled with your calculated numbers. After building it, if you have access to an oscilloscope or logic analyzer, capture the real waveform and annotate your sketch with the measured values next to your predicted ones.

Reflection

In one short paragraph, compare this lab’s multiplexed display to the naive alternative: wiring one segment-driver pin per segment per digit ($7 \text{ segments} \times 2 \text{ digits} = 14 \text{ pins}$, plus 2 more for the commons – more pins than the Uno has spare once the buttons and LEDs are wired). What did multiplexing buy you, and what did it cost – in firmware complexity (DIGIT_TO_SEG, the REFRESH loop, the dwell-time budget) or in extra components (two transistors, seven shared resistors) that a one-pin-per-segment design would not have needed? This exact circuit becomes a KiCad schematic and PCB in **Lab 4-2** – which of today’s breadboard mistakes, if any, do you expect would be harder to catch once it is a fabricated board instead of a breadboard you can freely rewire?